Geavanceerde computerarchitecturen - Tweede sessie

Cas Truyers en Niels Verhoeve

26 februari 2026

1 Introductie

In dit labo maken we een random array waaruit we de hoogste getalwaarde willen halen op basis van de CPU, GPU met atomic operation en GPU met reduction. Als controle printen we de array en hoogste getalwaarde (als realistisch), alsook de runtime van de verschillende functies. We starten van eenvoudige code met als doel later uit te breiden. De volledige code is te vinden op: https://github.com/Niels-V-KU/Lab_Geavanceerde_Computerarchitectuur/tree/main.

2 CPU

We starten van de CPU-code om dan het verschil in tijd tussen CPU en GPU operaties te meten. Dit is onze baseline voor de rest van de metingen. Een random 1D-array is gegenereerd met een instelbare lengte (minimale waarde nul, maximale waarde 32767). De code loops over de array en bewaart de grootste waarde.

```
    const auto start = std::chrono::steady_clock::now();

    for(int i = 0; i < sizeVec; i++){
        if(inVec[i] > max) max = inVec[i];
    }

    const auto end = std::chrono::steady_clock::now();
    const std::chrono::duration<double> elapsed_seconds{end-start};
```

3 GPU

3.1 Atomic operation

Atomic operation handelt de conflicten bij bewerkingen sequentieel af, en maakt dus minder gebruik van de voordelen van parallellisatie van de GPU, maar is in het algemeen flexibeler (bv. voor de grootte van de array) en geeft minder risico op fouten. Het zoeken van een maximum in een array is hier dus minder voor geschikt.

```
__global__ void atomic(int* maxVec, int sizeVec)
{
    int i=threadIdx.x+blockIdx.x*blockDim.x;
    if (i < sizeVec)
        atomicMax(&maxVec[0], maxVec[i]);
}
```

De verwachting is bijgevolg dat de atomic operation weinig winst gaat geven ten opzichte van de CPU.

3.2 Reduction

De reduction maakt veel meer gebruik van de parallellisatie en is daarom sneller, maar geeft ook meer mogelijkheden tot race condities en is minder flexibel in lengte waardoor voor lengtes die geen macht zijn van twee, additionele codering toegevoegd moet worden.

```
__global__ void reduction(int* maxVec, int sizeVec)
{
    int i=threadIdx.x+blockIdx.x*blockDim.x;
    int operations = sizeVec/2;
    int counter = 0;
    while(operations >= 1){
        int index = (1 << (counter+1)) * i;
        if (maxVec[index] < maxVec[index + (1 << counter)] && i < operations) maxVec[
    index] = maxVec[index + (1 << counter)];
        counter++;
        if (operations != 1) operations = operations/2; else operations = 0;
        // __syncthreads();
    }
}
```

De verwachting is dat zeker voor grotere arrays dit verschil duidelijk gaat worden omdat meer elementen parallel uitgevoerd kunnen worden en deze kleine besparingen samen een groter verschil maken.

3.3 Algemeen en uitbreiding

Deze zelf geschreven, relatief simpele functies werken goed voor kleinere arrays maar vertonen nog enkele fouten die verholpen kunnen worden met complexere code. Zo werkt de code niet meer correct voor grotere arrays, en werkt het programma niet correct als er minder beschikbare threads zijn dan elementen in de array.

Voor de kleinere arrays kunnen we wel al enkele vaststellingen maken, zo is de reductiemethode beter dan de atomic methode zoals voorspeld. De CPU is echter sneller. Het vermoeden is dat voor grotere arrays GPU methoden wel voordelig kunnen zijn. We zullen onze code dus moeten uitbreiden. Om de eerder vermelde problemen op te lossen hebben we een aantal extra principes toegepast die dit verhelpen, namelijk: grid-stride loops zodat de kernel werkt met willekeurige thread/data-verhoudingen, warp-shuffle instructies voor snellere communicatie zonder shared memory, een iteratieve reductie met meerdere kernel launches om elke invoergrootte (zelfs >2048) te verwerken, aparte buffers om originele data te behouden, correcte synchronisatie met syncthreads om race conditions te voorkomen, en flexibele configuratieopties voor het experimenteren met verschillende aantallen blocks en threads en een algemeen verder geoptimaliseerde code.

```
#define CUDA_CHECK(call) \
do { \
    cudaError_t err = call; \
    if(err != cudaSuccess){ \
        std::cerr << "CUDA error: " << cudaGetErrorString(err) \
                  << " at line " << __LINE__ << std::endl; \
        exit(EXIT_FAILURE); \
    } \
} while(0)

#define WARP_SIZE 32

// ============================================================
// ATOMIC MAX (grid-stride safe)
// ============================================================
__global__ void atomicMaxKernel(int* data, int* result, int n)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;

    for (int i = idx; i < n; i += stride)
        atomicMax(result, data[i]);
}

// ============================================================
// WARP REDUCTION
// ============================================================
__inline__ __device__
int warpReduceMax(int val)
{
    for (int offset = WARP_SIZE/2; offset > 0; offset /= 2)
```

```
        val = max(val, __shfl_down_sync(0xffffffff, val, offset));
    return val;
}

// ============================================================
// REDUCTION KERNEL
// ============================================================
__global__ void reductionMaxKernel(int* input, int* output, int n)
{
    extern __shared__ int sdata[];

    int tid = threadIdx.x;
    int globalIdx = blockIdx.x * (blockDim.x * 2) + threadIdx.x;

    int myMax = INT_MIN;

    if (globalIdx < n)
        myMax = input[globalIdx];

    if (globalIdx + blockDim.x < n)
        myMax = max(myMax, input[globalIdx + blockDim.x]);

    // warp reduction
    myMax = warpReduceMax(myMax);

    // write warp results
    if ((tid % WARP_SIZE) == 0)
        sdata[tid / WARP_SIZE] = myMax;

    __syncthreads();

    // final warp
    if (tid < blockDim.x / WARP_SIZE)
    {
        myMax = sdata[tid];
        myMax = warpReduceMax(myMax);

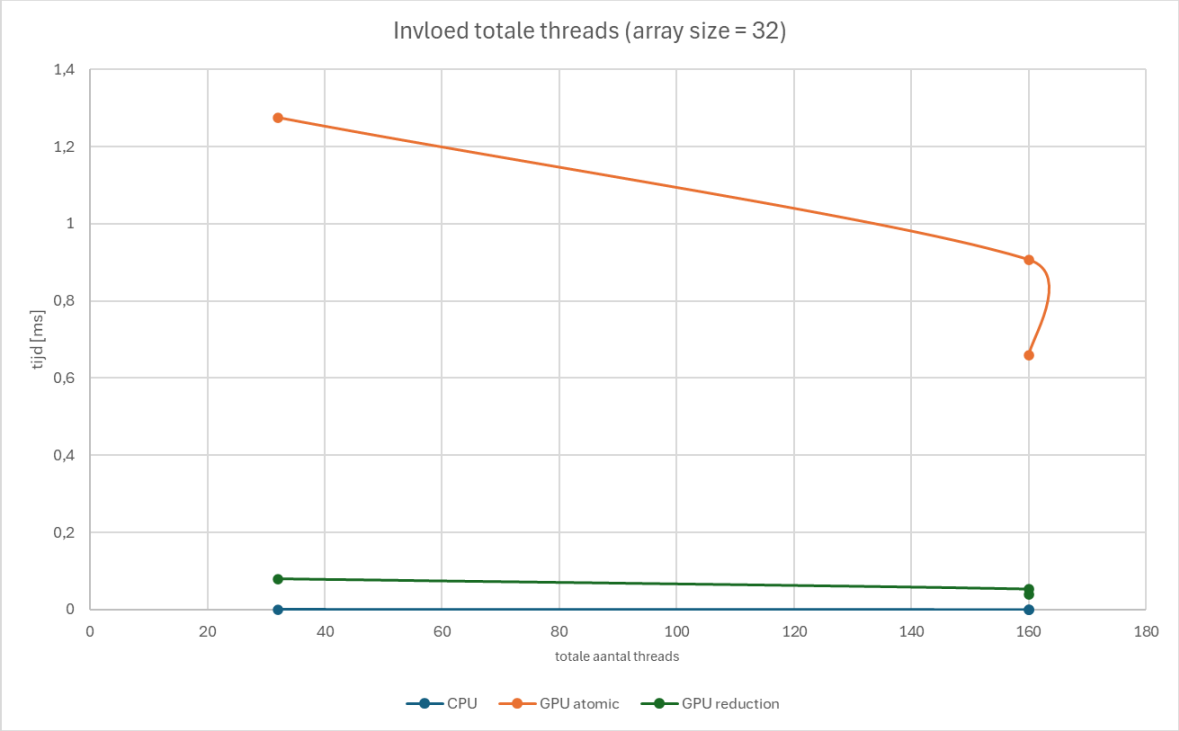
        if (tid == 0)
            output[blockIdx.x] = myMax;
    }
}

// ============================================================
// CPU MAX
// ============================================================
int cpuMax(int* data, int n)
{
    int maxVal = INT_MIN;
    for(int i=0;i<n;i++)
        if(data[i] > maxVal)
            maxVal = data[i];
    return maxVal;
}
```

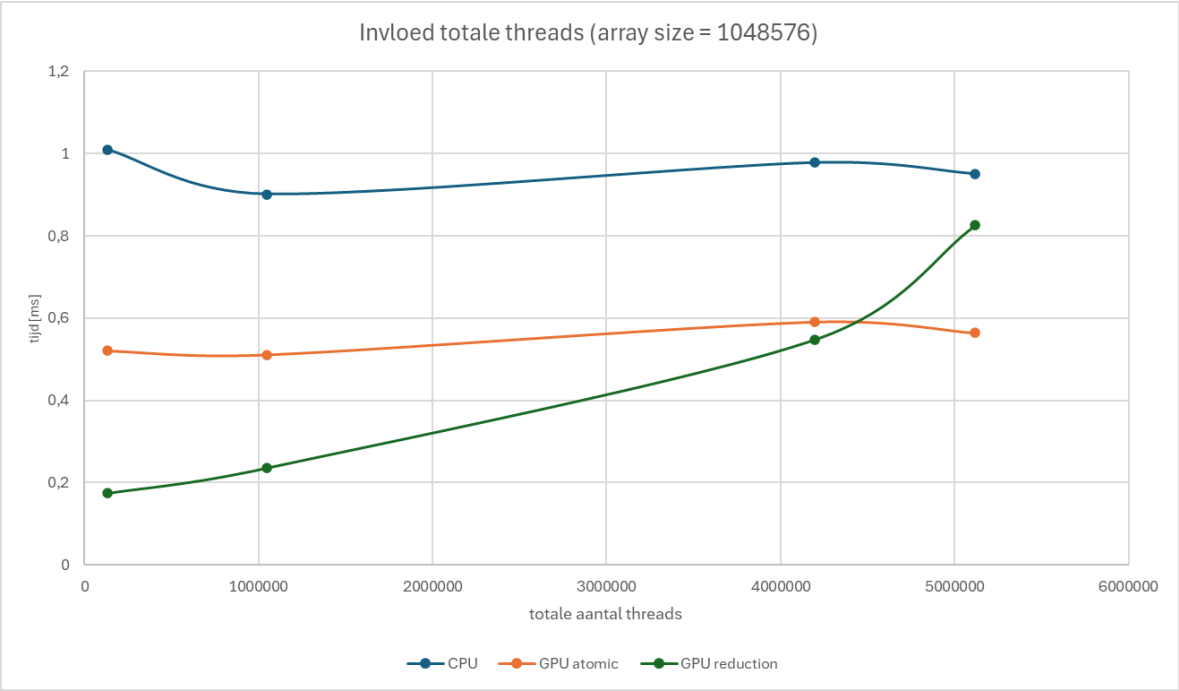
4 Metingen

We hebben de basis code gerund voor een array van 32 (2**5) elementen, en de uitgebreide code voor een array van 1048576 (2**20) elementen, verder naar verwezen als kleine en grootte array. Hierbij hebben we de block grootte en thread grootte laten variëren, en telkens de uitvoertijd gemeten (zonder geheugenoperaties en andere hulpfuncties). Het totale aantal threads is dan het aantal threads per block * het aantal blocks. Elke opgenomen waarde is een gemiddelde uit 6 metingen om een zo accuraat mogelijk resultaat te bekomen. Ook is er een speedup factor berekend (CPU_tijd / GPU_tijd). Deze waarden zijn uitgezet in onderstaande tabellen (alle metingen zijn in ms).

We zien dat voor de kleine array de reductie een minimale tot geen tijdswinst behaald voor een toenemend aantal threads. De atomic GPU operatie haalt wel een deel in tijdwinst, maar is nog altijd meer tijds consumerend dan de GPU met reductie en de CPU referentie. Dit is vermoedelijk omdat voor



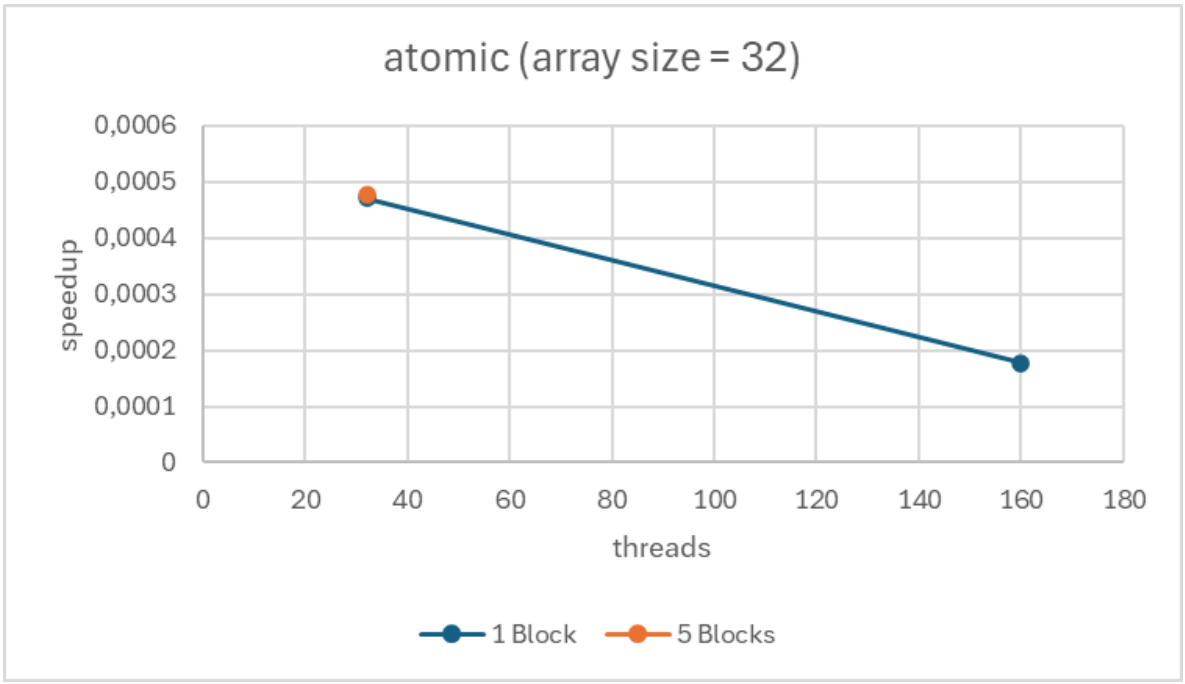
Figuur 1: Invloed van totale thread size op 32 element array



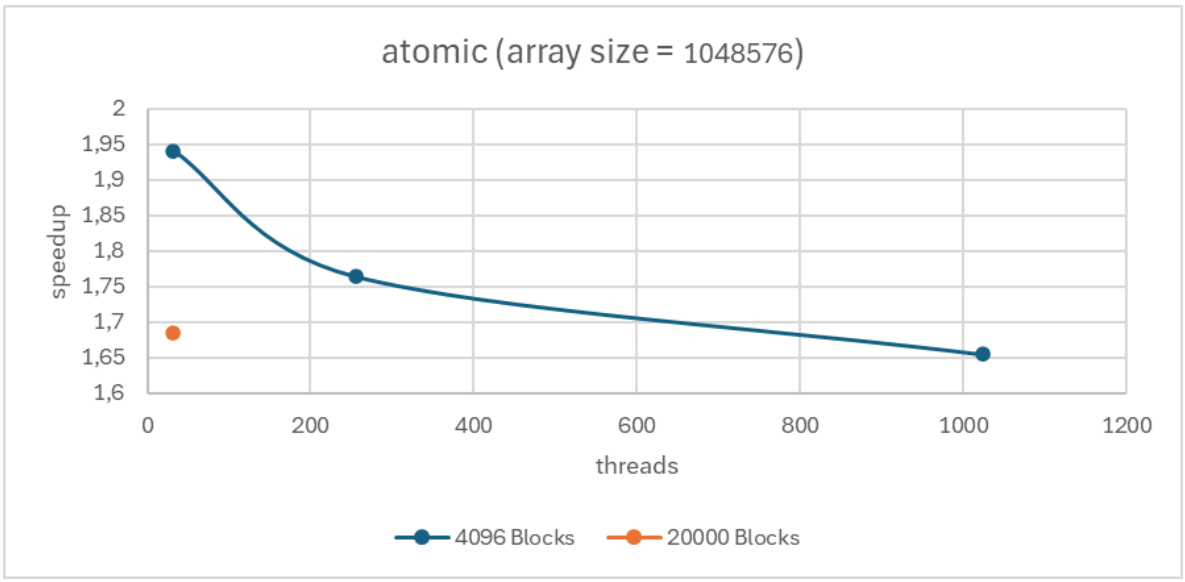
Figuur 2: Invloed van totale thread size op 1048576 element array

de kleine array de kernel launch overhead domineert ten opzichte van de tijdswinst op de bewerkingen, er moeten relatief weinig bewerkingen uitgevoerd worden. De grootte array vertoont meer fluctuaties. De CPU heeft nu het meest tijd nodig om de maximale waarde te filteren, het is dus duidelijk dat de GPU een belangrijke rol speelt als de taak meer word uitgevoerd. De GPU met reductie presteert echter beter voor een kleiner aantal totale threads dan voor het grotere aantal threads, de atomic

operation blijft quasi constant. Een groter aantal threads/blocks resulteert dus niet altijd in een beter resultaat. Dit is ook volgens ons verwachtingen, de overhead word meer verwaarloosd ten opzichte van de gewonnen tijd door het parallel uitvoeren van de sorteringsbewerkingen. Door de beperkte hardware (beperkt aantal SM's) is het aantal bruikbare threads beperkt. Als teveel threads gelanceerd worden wachten deze in een queue resulterend in geen tijdswinst en meer overhead. Daarnaast is er ook scheduling overhead waardoor een toenemend aantal blokken eenzelfde redenering volgt. Ook ons eerdere veronderstelling rond de atomic operation die serieel werkt, en weinig tijdswinst kan halen op deze manier, word hier bevestigd, het meest efficiënte algoritme voor onze toepassing met grootte arrays is de reductie. De verbeteringen in codeZ helpen ook om dit algoritme betrouwbaarder en sneller te maken.

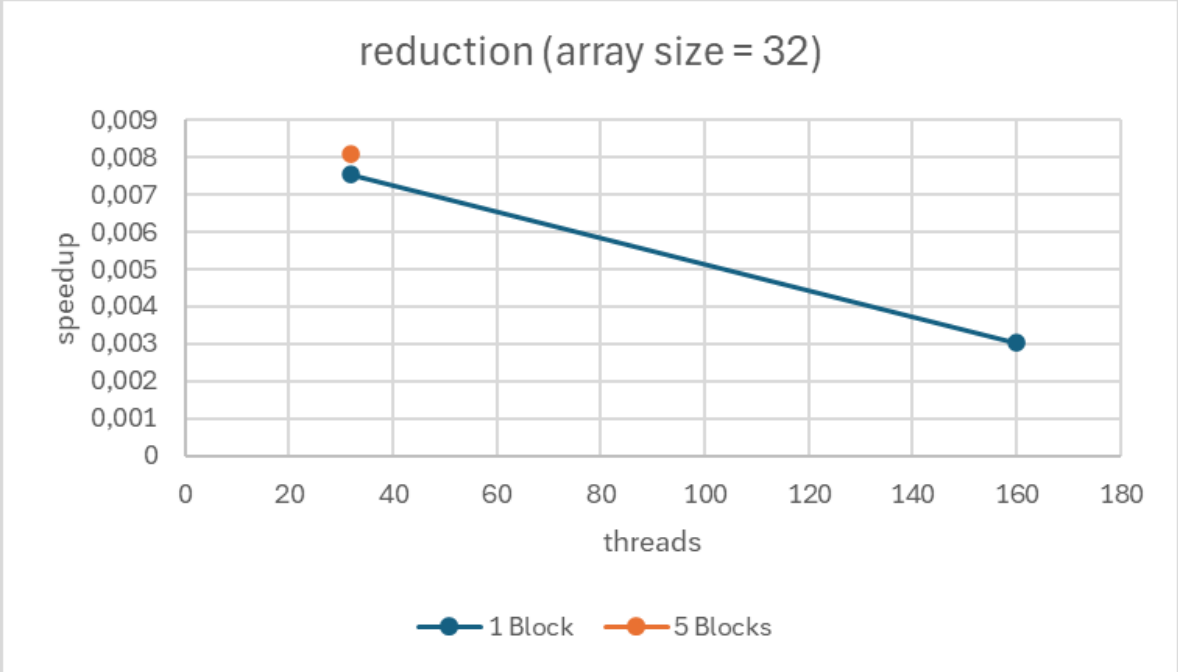


Figuur 3: Speedup factor Atomic - 32 element array

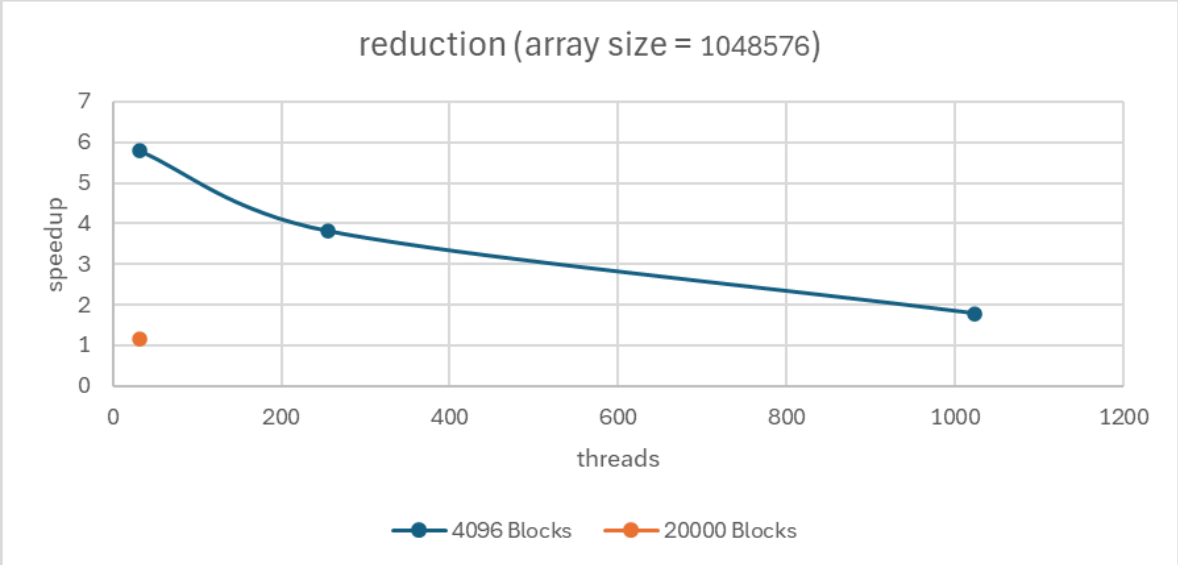


Figuur 4: Speedup factor Atomic - 1048576 element array

De speedup factor neemt af op de grafiek, maar slechts met een zeer kleine marge, dit kan dus evengoed aan de metingen liggen die lichtelijk variëren over meerdere metingen. We zien ook dat de meting voor 1 of 5 blocken zo goed als samenvallen voor de kleine array. Voor de grotere array zien we een grotere afwijking naarmate het aantal threads toeneemt, de speedup factor neemt af. We zien ook dat hier in tegenstelling tot de meeting met de kleine array er wel degelijk een verschil is tussen het aantal gebruikte blocks, meer blocks maken de speedup factor kleiner. Het is dus niet nuttig om hier meer blocken te gebruiken (dit kan ook liggen aan hardware beperkingen, zo is het aantal bruikbare blokken en threads beperkt, wat constraints oplegt).



Figuur 5: Speedup factor Reduction - 32 element array



Figuur 6: Speedup factor Reduction - 1048576 element array

Voor de reductie zien we dat voor de kleine array de waarden voor de speedup factor opnieuw ongeveer samen liggen voor 1 of 5 blokken. Ook neemt de speedup factor opnieuw af bij meer threads,

maar opnieuw met een heel kleine waarde. Voor de grootte array zien we nu een heel groot verschil voor toenemende threads, opnieuw neemt de speedup factor af. Ook meer blocks doet de speedup factor sterk afnemen. Deze conclusies zouden ook duidelijker worden met meer metingen, voor verschillende inputs en thread/block keuzes.

5 Conclusie

Het gebruik van de GPU kan, wanneer correct geïmplementeerd, zorgen voor een aanzienlijke tijdswinst bij sterk paralleliseerbare bewerkingen. Indien de implementatie echter onvoldoende rekening houdt met de eigenschappen van GPU-architecturen, kan dit juist leiden tot prestatieverlies ten opzichte van een CPU-implementatie. Uit de metingen blijkt dat een reductie-gebaseerde aanpak het meest efficiënt is voor operaties waarbij veel gelijkaardige berekeningen uitgevoerd worden, terwijl een atomic implementatie minder performant is door serialisatie van gelijktijdige geheugenoperaties.

Voor kleinere arrays en eenvoudigere berekeningen blijft de CPU doorgaans de betere keuze, aangezien de overhead van kernel launches en thread scheduling op de GPU relatief groot is. Daarnaast kan een foutieve implementatie leiden tot incorrecte resultaten. Belangrijke oorzaken hiervan zijn onder andere race conditions, foutief geheugenbeheer en onvoldoende synchronisatie tussen threads.

Bij toenemende inputgrootte stijgt de uitvoeringstijd, maar deze toename kan sterk beperkt worden door efficiënte parallellisatie. Door gebruik te maken van reductie en shared memory kan een sterke groei in probleemgrootte slechts een beperkte stijging in uitvoeringstijd veroorzaken. Verder kan het aanroepen van een kernel met onvoldoende threads ertoe leiden dat niet alle data-elementen verwerkt worden; dit probleem kan opgelost worden door het gebruik van grid-stride loops.

De aanpak is bovendien uitbreidbaar naar tweedimensionale arrays, waarbij het probleem kan worden opgesplitst in meerdere onafhankelijke deelproblemen die parallel verwerkt worden. Hierdoor kan ook voor grotere en complexere datasets een significante tijdswinst verwacht worden.

Referenties